ENDIVE-CLOUD

멀티리전·하이브리드 클라우드 기반 야외행사 안내 플랫폼

목차

01 프로젝트 개요

02 프로젝트 팀 구성 역할

03 프로젝트 수행 절차 및 방법

04 프로젝트 수행 경과

05 자체 평가 의견

프로젝트 개요

선정 배경

디지털 인프라 장애의 사회적 파급력 확대

정부·금융·게임 등 다양한 서비스가 디지털 인프라로 연결됨
⇒ 한 지점의 장애가 사회 전반의 서비스 중단으로 확산될 가능성 증가

단일 데이터센터·클라우드 의존 구조의 한계

특정 데이터센터의 화재나 클라우드 리전 장애가 다수 시스템에 동시에 영향을 미침
⇒ 장애 영역을 분리한 이중화와 원격지 재해복구 체계 필요

비용과 운영 복잡성을 고려한 DR 전략 필요

멀티리전·멀티클라우드는 안정성을 높이지만 인프라 관리 부담도 함께 증가
⇒ 시스템 중요도를 고려한 현실적인 복구 전략 필요



프로젝트 개요

활용 방안 및 기대효과

01

행사 정보 통합 플랫폼으로 활용

여러 공공데이터에 분산된 행사 정보를 수집·가공하여 일정, 위치 및 위험 정보를 한 곳에서 제공

지역 행사·문화 정보 서비스로 확장하여 사용자의 정보 탐색 편의성과 데이터 활용성을 향상

공공데이터 | 행사 정보 | 위험 분석 | 웹 서비스

02

클라우드 기반 운영 표준 모델로 활용

서비스 기능을 컨테이너로 분리하고 Kubernetes 환경에서 독립적으로 배포·확장할 수 있는 운영 구조 적용

동일한 구성 방식을 다른 웹 서비스에도 재사용하여 구축 과정과 운영 복잡도 및 수작업 오류를 감소

Container → Kubernetes → 구성 재사용 → 운영 일관성

03

멀티리전 DR복구를 통한 서비스 연속성 향상

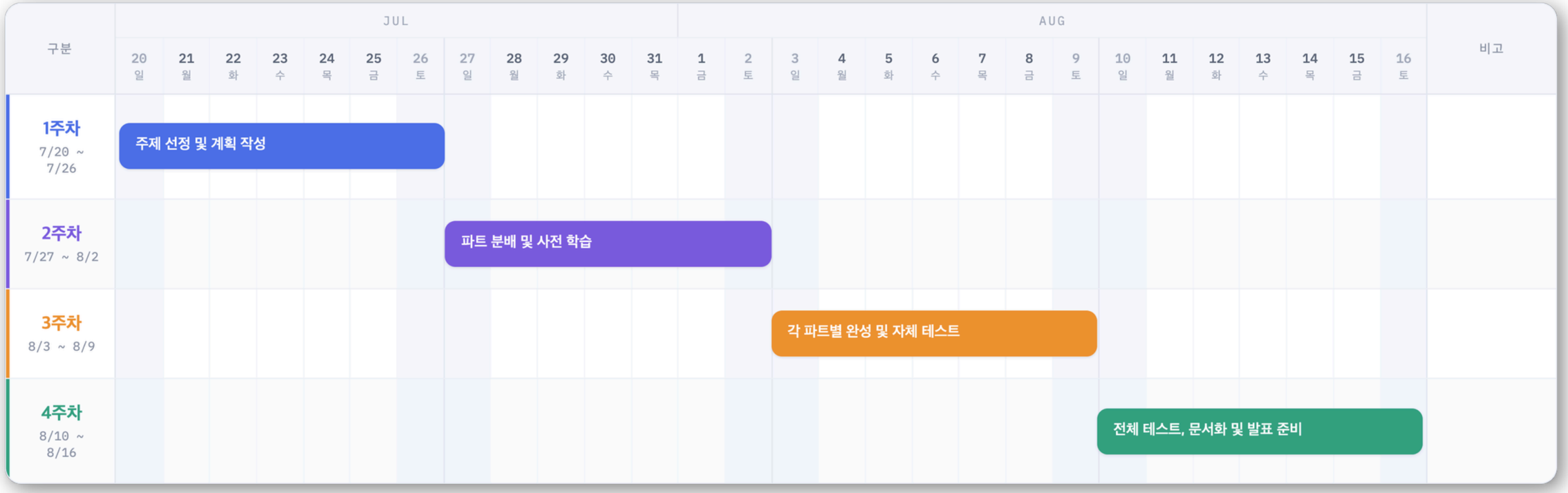
서울 리전 전체 장애 시 지리적으로 분리된 도쿄 리전에 인프라와 애플리케이션을 DR복구

복구 절차를 표준화하고 반복 검증하여 장애에 따른 서비스 중단 시간과 운영자의 대응 부담을 감소

서울 운영 | 도쿄 DR복구 | 장애 영역 분리 | 복구 검증

프로젝트 개요

프로젝트 절차 및 수행 방법



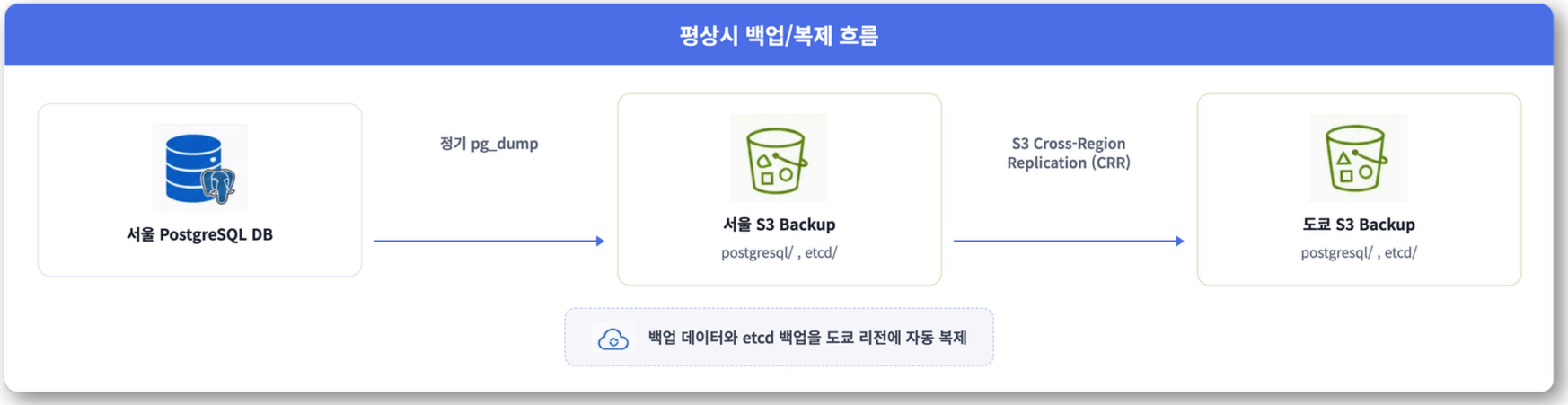
프로젝트 팀 구성 및 역할

프로젝트 팀 구성 및 역할

이름	역할	담당 업무	
최진성	팀장	Event API	프론트엔드
유준형	팀원	쿠버네티스	Ansible
이주원	팀원	CI/CD	모니터링
이주희	팀원	데이터 수집	Risk Engine
조민석	팀원	인프라	Terraform

프로젝트 수행 경과

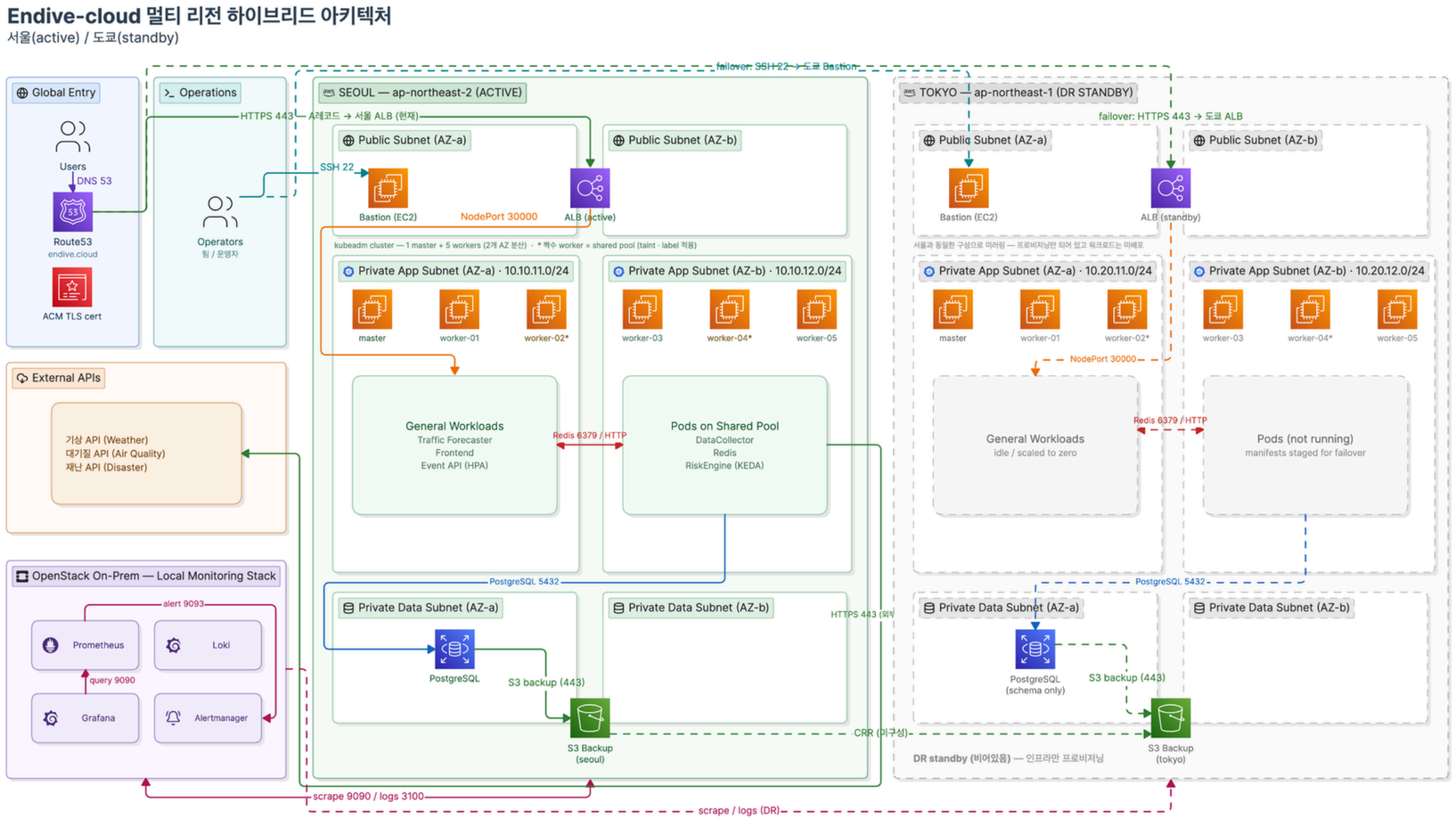
시연영상 시나리오





프로젝트 수행 경과

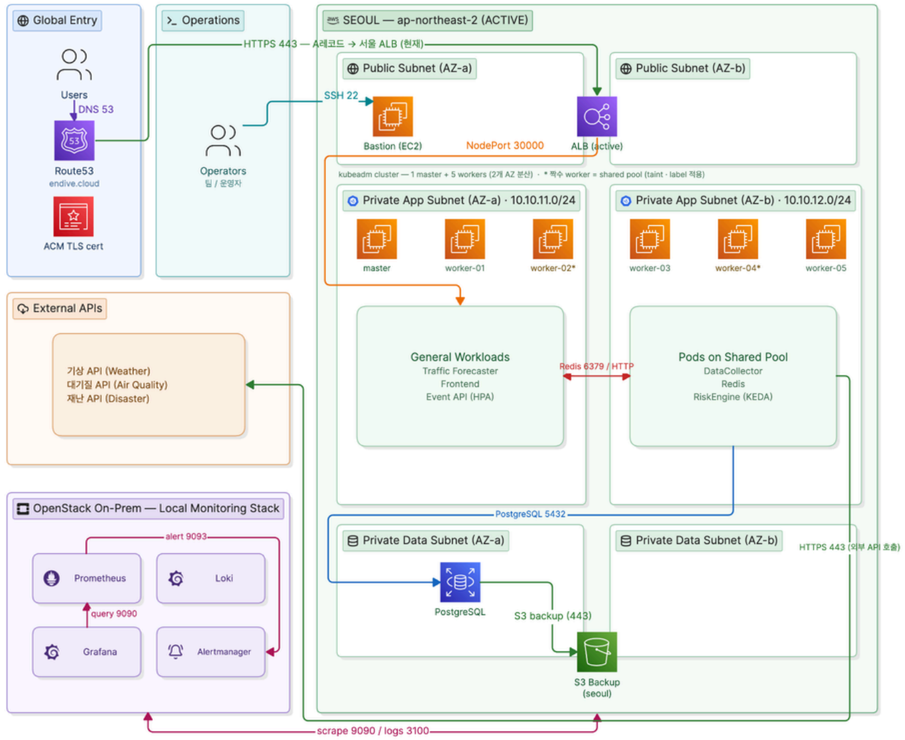
인프라 아키텍처



- 서울 리전을 주 운영 환경으로 구성하고 도쿄 리전을 DR 환경으로 구축
- Terraform 기반 IaC로 네트워크·EC2·보안·스토리지 인프라 자동화
- S3·ECR Cross-Region Replication으로 DB 백업 및 컨테이너 이미지 복제
- 장애 발생 시 Jenkins·Ansible을 통해 도쿄 환경으로 복구 가능한 DR 체계 구성
- OpenStack 환경의 Jenkins·Prometheus·Grafana와 AWS 인프라를 연계하여 배포·복구·모니터링 수행

프로젝트 수행 경과

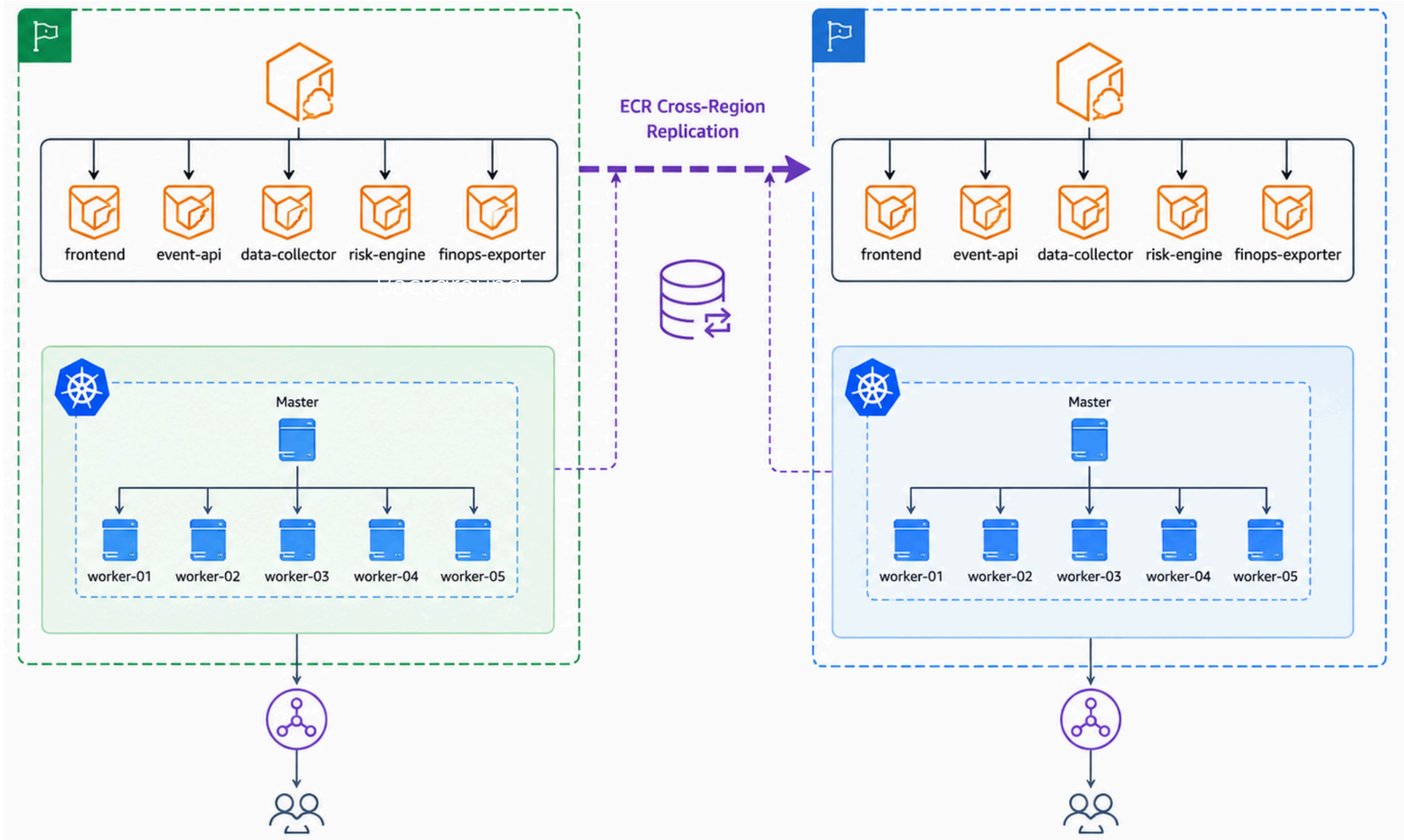
AWS 서울 리전 서비스 아키텍처



- VPC 10.10.0.0/16 기반 Public / Private App / Private Data 네트워크 분리
- Bastion을 통해 Private 영역의 Kubernetes 및 PostgreSQL 서버에 접근
- Kubernetes Master 1대 + Worker 5대로 애플리케이션 실행 환경 구성
- PostgreSQL DB는 Private Data Subnet에 배치하여 외부 직접 접근 차단
- ECR·S3·IAM을 연계하여 이미지 관리 및 DB/etcd 백업 환경 구성
- Terraform 모듈화를 통해 서울 인프라 생성·변경·재구성 자동화

프로젝트 수행 경과

ECR Cross – Region Replication



- 서울 리전에 Private ECR Repository 5개 구성
- endive/* 이미지를 서울 → 도쿄 ECR로 자동 복제
- 도쿄 DR 환경에서 별도 이미지 재빌드 없이 동일 이미지 Pull 가능
- 실제 event-api 테스트 이미지로 서울·도쿄 동일 Digest 복제 확인
- 이미지 IMMUTABLE / Scan on Push / AES256 / Lifecycle Policy 적용

프로젝트 수행 경과

Terraform IaC 구성

```
terraform/
├── modules/
│   ├── network
│   ├── ec2
│   ├── security
│   ├── iam
│   ├── s3
│   └── ecr
│
└── environments/
    ├── hot-seoul
    ├── cold-tokyo-persistent
    └── cold-tokyo-runtime
```

- Network · EC2 · Security · IAM · S3 · ECR을 Terraform Module로 구성
- 공통 Module을 활용해 서울 / 도쿄 멀티리전 인프라 관리
- 서울은 Active 환경, 도쿄는 DR 환경으로 분리
- 도쿄는 Persistent / Runtime으로 분리하여 DR 자원 관리
- Plan / Apply / Destroy를 통해 인프라 생성·변경·재구성 자동화

Persistent	S3 · ECR · IAM · Key Pair → Runtime 삭제 후에도 유지
Runtime	VPC · Subnet · NAT · SG · EC2 → DR 발생 시 Terraform으로 재생성

프로젝트 수행 경과

Ansible playbook 구성 – 서울(active)

ansible 구간의 모든 동작은 동적 인벤토리를 기반으로 동작

main.yml

```
├─ playbooks/bastion_connect.yml
├─ playbooks/cluster_settings.yml
│  ├─ playbooks/components/common.yml
│  ├─ playbooks/components/master.yml
│  └─ playbooks/components/worker.yml
├─ playbooks/addons_settings.yml
│  ├─ playbooks/components/helm.yml
│  ├─ playbooks/components/ingress.yml
│  ├─ playbooks/components/storageclass.yml
│  ├─ playbooks/components/ecr_credential_provider.yml
│  ├─ playbooks/components/tailscale_operator.yml
│  ├─ playbooks/components/argocd.yml
│  └─ playbooks/components/demo_pages.yml
├─ playbooks/db_settings.yml
│  └─ playbooks/components/db_set-up.yml
└─ playbooks/backup_settings.yml
   ├─ playbooks/components/db_backup.yml
   └─ playbooks/components/etcd_backup.yml
```

- bastion_connect.yml: bastion 노드를 등록 후 접속하여 k8s cluster 구축을 위한 control node로 setting
- cluster_settings: common.yml 에서 master, worker 노드들의 호스트네임 및 ssh 접속 정보 수집 후 kubeadm 사전 세팅 준비(swap off 등)를 한 후 각각 master, worker.yml 을 통해 k8s cluster를 setting(짝수 노드의 경우, 고 메모리 pod를 위한 taint 부착)
- addons_settings: helm, nginx-ingress, storageclass 등 각종 애드온을 k8s cluster 내에 설치 및 배포
- db_settings: db node에 Postgresql 설치, db 생성 및 연결
- backup_settings: Postgresql의 db와 k8s cluster etcd의 스냅샷을 S3로 백업 시키는 setting(cron으로 동작하며, 별도의 변수 처리로 사용자 임의 지정)

프로젝트 수행 경과

Ansible playbook 구성 – 서울(active)

ansible 구간의 모든 동작은 동적 인벤토리를 기반으로 동작

monitoring.yml

```
├ playbooks/bastion_connect.yml
└ playbooks/monitoring_settings.yml
    ├ playbooks/components/node_tailscale.yml
    ├ playbooks/components/aws_monitoring.yml
    └ playbooks/components/aws_logging.yml
```

recovery.yml

```
├ playbooks/bastion_connect.yml
└ playbooks/recovery_settings.yml
    ├ playbooks/components/argocd_repository.yml
    ├ playbooks/components/app_secrets.yml
    └ playbooks/components/argocd_application.yml
```

alb-up.yml

```
└ playbooks/alb_up_settings.yml
```

alb-delete.yml

```
└ playbooks/alb_delete_settings.yml
```

- monitoring.yml: 노드를 tailnet에 가입시켜 경로를 연 뒤 kube-prometheus-stack과 Alloy를 설치(메트릭·로그는 OpenStack 중앙 서버로 전송하며, 클러스터 내부에는 Grafana를 두지 않음)
- recovery.yml: argocd_repository로 레포 인증 Secret 생성 → app_secrets로 앱 환경변수 주입 → argocd_application으로 Application 등록(최초 씨앗만 심으면 이후 배포는 Argo CD가 자동 동기화)
- alb-up.yml: ALB·타겟그룹을 생성해 워커 NodePort에 연결하고, ACM 인증서로 HTTPS를 종단한 뒤 Route53 A레코드를 연결(베스천을 거치지 않고 AWS API를 직접 호출)
- backup.yml: main에서 이미 세운 백업 cron의 주기만 변경할 때 사용하는 별도 진입점
- main과 진입점을 나눈 이유: 앱 레포 상태, 토큰 만료 등 통제 밖 조건에 의존하므로, 외부 사정으로 클러스터 재구성 자체가 실패하지 않게 분리

프로젝트 수행 경과

Ansible playbook 구성 – 도쿄(standby)

ansible 구간의 모든 동작은 동적 인벤토리를 기반으로 동작

dr-main.yml

```
└ main.yml (dr_target_region: tokyo)
  ├ playbooks/bastion_connect.yml
  ├ playbooks/cluster_settings.yml
  │  ├ playbooks/components/common.yml
  │  ├ playbooks/components/master.yml
  │  └ playbooks/components/worker.yml
  ├ playbooks/addons_settings.yml
  │  ├ playbooks/components/helm.yml
  │  ├ playbooks/components/ingress.yml
  │  ├ playbooks/components/storageclass.yml
  │  ├ playbooks/components/ecr_credential_provider.yml
  │  ├ playbooks/components/tailscale_operator.yml
  │  ├ playbooks/components/argocd.yml
  │  └ playbooks/components/demo_pages.yml
  ├ playbooks/db_settings.yml
  │  └ playbooks/components/db_set-up.yml
  └ playbooks/backup_settings.yml
     ├ playbooks/components/db_backup.yml
     └ playbooks/components/etcd_backup.yml
```

dr-monitoring.yml

```
└ monitoring.yml (dr_target_region: tokyo)
  ├ playbooks/bastion_connect.yml
  └ playbooks/monitoring_settings.yml
     ├ playbooks/components/node_tailscale.yml
     ├ playbooks/components/aws_monitoring.yml
     └ playbooks/components/aws_logging.yml
```

dr-recovery.yml

```
└ recovery.yml (dr_target_region: tokyo)
  ├ playbooks/bastion_connect.yml
  └ playbooks/recovery_settings.yml
     ├ playbooks/components/argocd_repository.yml
     ├ playbooks/components/app_secrets.yml
     └ playbooks/components/argocd_application.yml
```

dr-alb-up.yml

```
└ alb-up.yml (tokyo)
   └ playbooks/alb_up_settings.yml
```

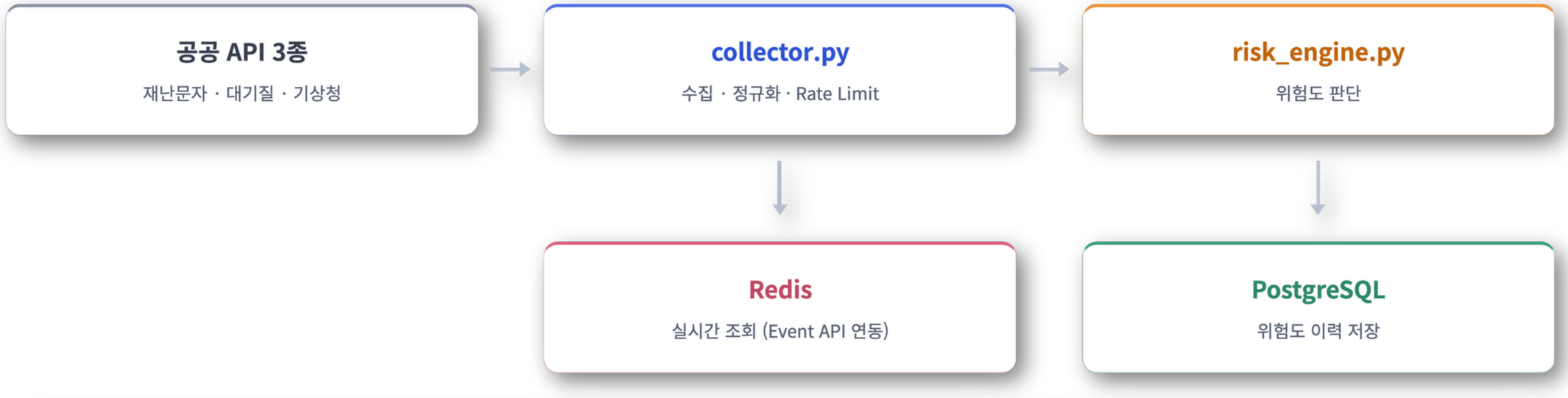
dr-alb-delete.yml

```
└ alb-delete.yml (tokyo)
   └ playbooks/alb_delete_settings.yml
```

- dr-*.yml: 서울과 같은 플레이북을 사용하며, 변수로 dr_target_region=tokyo만 주입
- dr-alb-up.yml: 해당 플레이북을 실행할 경우, endive.cloud의 주소가 도쿄 ALB를 디렉팅하며, 실제 트래픽이 전환

프로젝트 수행 경과

전체 데이터 파이프라인

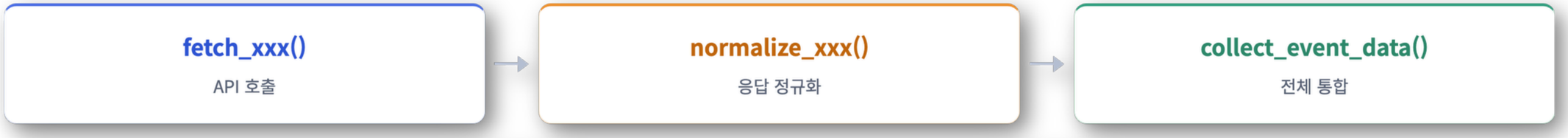


scheduler.py | 위 전체 흐름을 반복 실행 — 행사까지 남은 시간과 위험도에 따라 수집 주기를 자동 조절 (6시간 → 1시간 → 15분 → 5분 → 2분)

프로젝트 수행 경과

Data Collector: 공공 API 3종을 하나로 묶는 파일

재난문자 · 대기질 · 기상청 API를 호출해, 서로 다른 응답 형식을 하나로 통일



RateLimitCounter	API별 하루 호출 횟수를 세고, 한도 초과 시 자동 차단
fetch_disaster_alerts / fetch_air_quality / fetch_weather	각 API 실제 호출
normalize_disaster_alerts / normalize_air_quality / normalize_weather	응답을 정해진 필드명으로 변환
collect_event_data()	위 모든 함수를 순서대로 호출해 최종 통합 결과 반환

프로젝트 수행 경과

Risk Engine: 위험도는 어떻게 결정되는가

대기질 · 강수확률 · 재난문자, 세 지표를 각각 판정한 뒤 “가장 심각한 것”을 최종 위험도로 채택

대기질 (grade)

- 1~2 → 정상
- 3 → 주의
- 4 이상 → 위험

강수확률 (%)

- 50% 미만 → 정상
- 50~79% → 주의
- 80% 이상 → 위험

재난문자 (level)

- 없음 → 정상
- 일반 발령 → 주의
- “위급/경계/심각” → 위험

종합 판정 규칙 | 세 지표 중 가장 심각한 등급을 그대로 “종합 위험도”로 채택. 예) 대기질·날씨 정상 + 재난문자 발령 → 종합 = 주의

assess_overall_risk() _worse()로 더 심각한 쪽을 남기며 최종 등급을 좁혀감

프로젝트 수행 경과

Redis 저장: 실시간 최신 상태 캐싱

Event API가 화면에 즉시 보여줄 “지금 이 순간”의 위험도를, 빠르게 조회 가능한 Redis에 저장

```
key = "event:서울_한강여름플리마켓:snapshot"
try:
    redis_client.set(key, json.dumps(result, ensure_ascii=False))
    print(f"Redis에 저장 완료: {key}")
except redis.exceptions.ConnectionError:
    print("⚠ Redis 서버에 연결할 수 없습니다 (아직 서버가 준비되지 않았을 수 있어요).")
```

키 형식: event:{행사ID}:snapshot	행사마다 하나의 키를 갖고 최신 값으로 덮어씀
try/except	Redis 서버가 꺼져도 전체 파이프라인은 멈추지 않게 함

프로젝트 수행 경과

PostgreSQL 저장: 위험도 이력 기록

Redis와 달리, 매번 새로운 행(row)으로 쌓아 “위험도가 어떻게 변해왔는지” 시간순 기록을 함

```python
def save_risk_snapshot(session, event_id: int, collected: dict, risk: dict):
    """collect_event_data()와 assess_overall_risk() 결과를 DB에 저장"""
    snapshot = EventRiskSnapshot(
        event_id=event_id,
        overall_risk=risk["overall_risk"],
        risk_weight=risk["risk_weight"],
        air_quality_risk=risk["breakdown"]["air_quality"],
        weather_risk=risk["breakdown"]["weather"],
        disaster_risk=risk["breakdown"]["disaster"],
        raw_data=collected,
    )
    session.add(snapshot)
    session.commit()
    return snapshot
```

session.add	저장할 준비 (DB에 반영 안됨)
session.commit	실제로 저장 (DB에 새 행이 생김)

프로젝트 수행 경과

Event API 및 프론트엔드 / 서비스 연동 및 파드화

```
ubuntu@seoul-master-01:~$ kubectl get pod
NAME                                READY   STATUS    RESTARTS      A
data-collector-77854d4f7f-kkzlr     1/1     Running   4 (18h ago)   1
event-api-559bc7c77f-p5b7f          1/1     Running   0             2
redis                               1/1     Running   0             1
risk-engine-b777d548f-sbnq8         1/1     Running   0             2
ubuntu@seoul-master-01:~$
```

```
INFO:main:[PUB/SUB] 'CREATE' 이벤트 발행 완료 : event_129 (지역 : 부산 , Land ID: 11H20000)
INFO:     10.244.64.66:42778 - "POST /admin/events HTTP/1.1" 303 See Other
INFO:     10.244.64.66:42778 - "GET /view/admin/events HTTP/1.1" 200 OK
INFO:     10.244.64.66:42786 - "GET /view/admin/event/event_129 HTTP/1.1" 200 OK
INFO:     10.244.64.66:42786 - "GET /api/v1/admin/events/event_129/qr HTTP/1.1" 200 OK
INFO:     10.244.64.66:42786 - "GET /view/admin/event/event_128 HTTP/1.1" 200 OK
INFO:     10.244.64.66:42778 - "GET /api/v1/admin/events/event_128/qr HTTP/1.1" 200 OK
INFO:     10.244.64.66:35506 - "GET /view/admin/events/new HTTP/1.1" 200 OK
INFO:main:[EVENT CREATED] 신규 행사 생성 : event_130 (한강페스티벌 ) - 지역 : 서울
INFO:main:[DB SYNC] PostgreSQL events 테이블에 행사 등록 완료 (ID: 130, 지역명 : 서울특별시 , 시도명
INFO:main:[PUB/SUB] 'CREATE' 이벤트 발행 완료 : event_130 (지역 : 서울 , Land ID: 11B00000)
INFO:     10.244.64.66:48968 - "POST /admin/events HTTP/1.1" 303 See Other
INFO:     10.244.64.66:48968 - "GET /view/admin/events HTTP/1.1" 200 OK
INFO:     10.244.64.66:48968 - "GET /view/admin/event/event_130 HTTP/1.1" 200 OK
```

```
[Debug] 행안부 API 키 로드 상태 : True
[Debug] 중기예보 API 키 로드 상태 : True
[Debug] 단기예보 API 키 로드 상태 : True
[Scheduler] 백그라운드 주기적 폴링 스레드가 시작되었습니다.
[Scheduler] 다음 자동 갱신까지 대기 중... (예정 시각 : 2026-08-14 08:03:00)
[Data Collector Service] Running... Waiting for 'event_lifecycle_events' & Background Polling
[Collector] 데이터 수집 시작 (ID: event_125)
[Disaster API Response] Status: 200, Body: {"header":{"resultMsg":"NORMAL SERVICE","resultCode":"00","errorMsg":null}
[Disaster] 수집된 재난 문자 개수 : 3
[Debug] 넘어온 sido_name: '부산' (길이 : 2)
[Air API Response] Status: 200, Body: {"response":{"body":{"totalCount":36,"items":[{"pm25Grade1h":"1","pm10Value24":
[Air Quality] 측정소 : 개금동 , 미세먼지(PM10): 18, 초미세먼지(PM2.5): 6
[Debug] 행사 시작일 : 2026-08-14 16:55:00, 현재 시각 : 2026-08-14 07:55:44.992330, 남은 시간 (초): 32355.00767
[Collector] 단기예보 API 호출 (D-3 이내 - 지역 : 부산광역시 )
[Short-term Query] base_date: 20260814, base_time: 0500, nx: 60, ny: 127
[Short-term API] Status: 200
[Short-term API] 가져온 아이템 개수 : 300
[Short-term Parsed] 관측소 : 부산광역시 (격자 관측 ), 최저기온 : 24.0, 최고기온 : 33.0, 강수확률 : 30%
[Collector] Redis Snapshot 저장 완료 (Fallback 적용 ): event:event_125:snapshot
[Collector] Risk Engine으로 수집 완료 이벤트 발행 : collector_done
```

분산된 서비스의 파드(Pod) 통합 운영

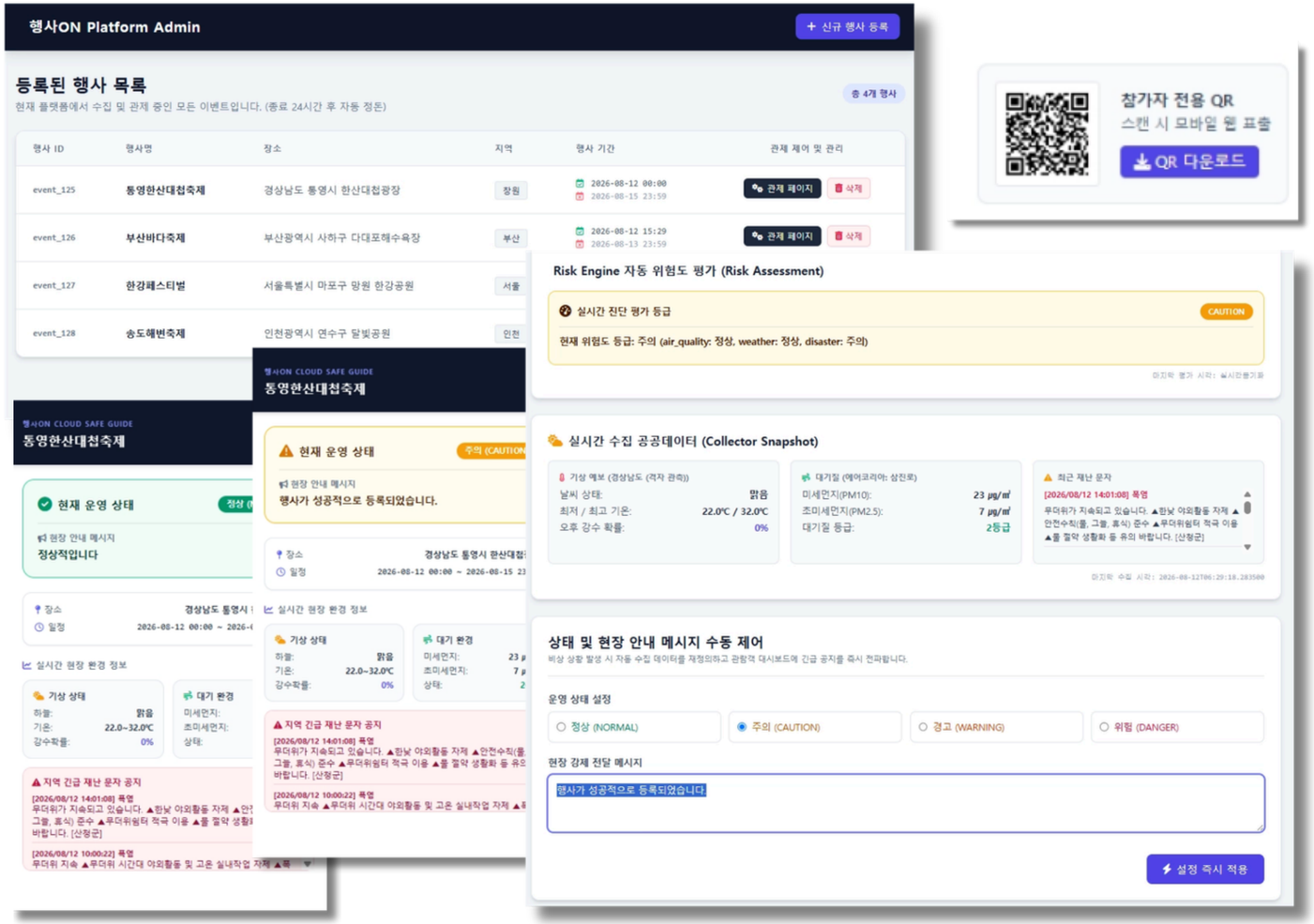
- Collector, Risk Engine, DB/Cache 인프라를 독립적인 4개의 파드로 컨테이너화
- 각 서비스의 생명주기를 격리하여 장애 전파 방지 및 수평 확장(Scale-out) 기반 마련

API Gateway 중심의 서비스 파이프라인

- FastAPI를 중앙 게이트웨이로 활용하여, 분산된 백엔드 파드들의 데이터를 사용자 요청에 맞춰 실시간 통합·중계
- Jinja2 엔진을 통해 실시간 데이터를 대시보드 및 모바일 뷰로 즉각 렌더링

프로젝트 수행 경과

Event API 및 프론트엔드 / 서비스 연동 및 파드화 : Frontend



사용자 접점 UI/UX 실시간 연동

- 행사 생성 즉시 QR 코드 자동 생성 API 연동 (사용자 접근성 강화)
- 참가자 실시간 모바일 뷰: 현재 행사 위치와 리스크 상황을 모바일에서 즉시 조회

통합 관제 인터페이스 개발

- 관리자용 통합 대시보드: 수집된 모든 데이터의 조회, 수정, 긴급 알림(Override)을 단일 페이지에서 수행

기술 비교 및 선택 : FastAPI

- Django/Flask vs FastAPI: Django의 무거운 빌트인 기능이나 Flask의 동기식 구조 대비, FastAPI는 비동기(Async) 기반의 초경량 구조로 고성능 API 처리에 최적화됨.
- 제한된 쿠버네티스 워커 자원 환경에서 최소 메모리 점유로 최대 퍼포먼스를 내기 위해 최적의 선택.

프로젝트 수행 경과

Event API 및 프론트엔드 / 서비스 연동 및 파드화 : Troubleshooting

데이터 정합성 최우선 (DR 복구시)	**문제** 메모리(Redis/events_db) 데이터 휘발성. **해결** 시스템 부팅 시(Startup) PostgreSQL DB를 최우선으로 호출(Re-hydration)하여 메모리 상태를 즉시 복구, 멀티 리전 DR 시에도 최대한 데이터 연속성 유지.
외부 API 갱신 주기 대응	**문제** 1시간 주기의 공공 데이터 갱신과 사용자 실시간 요청 사이의 간극. **해결** Redis Look-Aside 캐싱을 통해 외부 API 갱신 전까지는 캐시 데이터로 응답속도를 유지하고, 갱신 시점(1시간)에 데이터를 최신화하여 실시간 서비스 성능 보장.
DB 무결성 유지 (Cascading)	**문제** DB 참조 무결성 제약 조건(Foreign Key) 위반. **해결** 자식 테이블(Snapshot) 선 삭제 후 부모 테이블(Event)을 삭제하는 Atomic 삭제 로직 구현으로 운영 환경의 데이터 오류 원천 봉쇄.

프로젝트 수행 경과

한눈에 보는 CI/CD 흐름

GitOps 기반 멀티 리전 배포 구조

1 코드 변경

GitHub · endive-service

소스 수정 / PR

- feature 브랜치 작업
- PR 검토 후 main 병합
- Webhook으로 Jenkins 실행

소스 + Jenkinsfile + GitOps

2 CI

Jenkins Multibranch

1. 변경 범위·필수 파일 검증
2. 이미지 태그 생성
3. Docker 이미지 3종 빌드
4. FastAPI 헬스체크

feature · 검증만

main · ECR Push

3 이미지·GitOps

Registry

서울 ECR Push
↓
도쿄 ECR 복제

GitOps

seoul / tokyo Overlay의 newTag 갱신

빌드번호-커밋해시

CD 커밋 생성
예: 13-b213141

4 CD

Argo CD

- Git main 변경 감지
- Automated Sync
- Self Heal · Prune

서울 · overlays/seoul

도쿄 · overlays/tokyo

Healthy / Synced

5 배포

Kubernetes

서울 · Active

event-api
data-collector
risk-engine · redis
Ingress · Secret

도쿄 · DR 복구

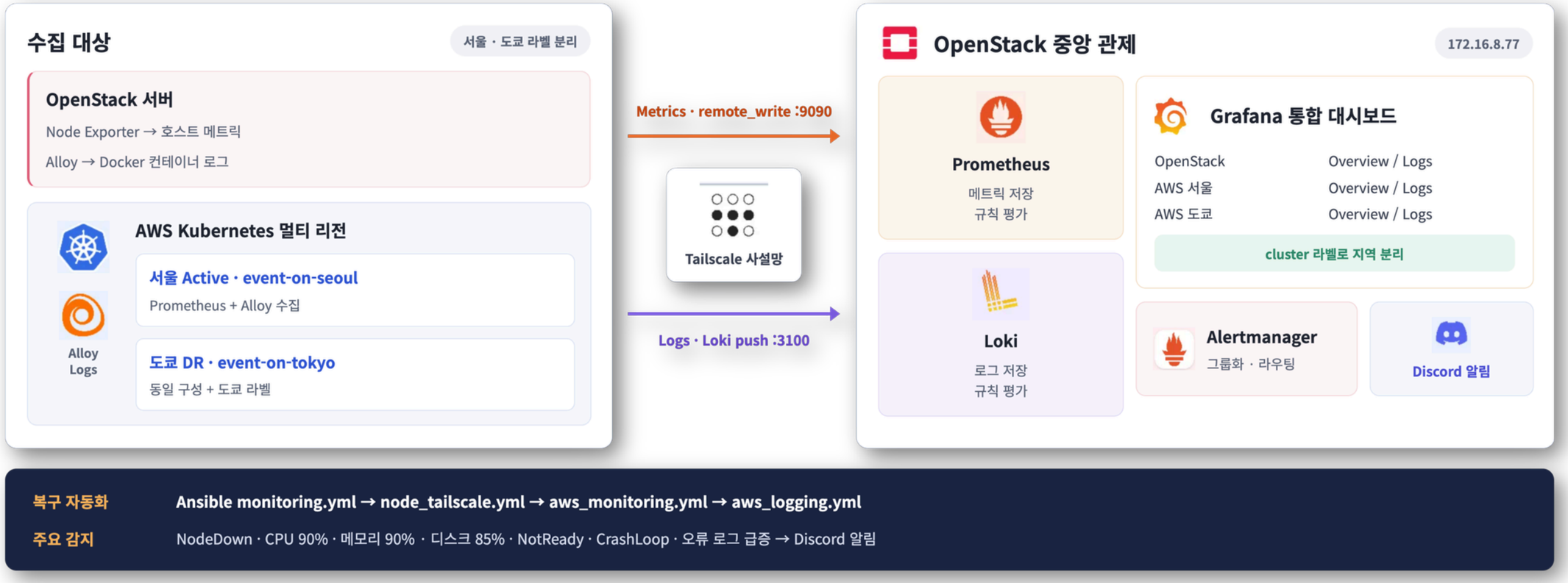
기존 최신 이미지로
재빌드 없이 배포

핵심 원칙 | Jenkins는 Kubernetes에 직접 배포하지 않는다 → Git을 갱신하고 Argo CD가 배포한다

프로젝트 수행 경과

한눈에 보는 통합 모니터링 구조

OpenStack 중앙 관제, AWS 서울 Active / 도쿄 DR



자체 평가 의견

추후 개선 및 보완 방향

제어 평면 고가용성

단일 마스터 구조를 다중
Control Plane과 etcd
이중화 구조로 개선

Secret 관리 강화

Vault 암호화·키 순환·
External Secrets 적용으로
민감정보 보호

DR 전환 자동 검증

복구 후 노드·Pod·서비스·
데이터 정합성을 자동 점검

멀티 리전 배포 고도화

ECR 복제와 GitOps
리전 전환·롤백 절차를
완전 자동화

통합 모니터링 안정성

중앙 관측 서버 백업과
알림 기준 세분화로
탐지 신뢰도 향상

정기 복구 훈련

Runbook을 표준화하고
반복 훈련으로 실제
대응 역량 강화

자동화 · 보안 · 고가용성 · 운영 표준화

자체 평가 의견

최진성



계획한 일정과 프로젝트 범위를 무탈하게 잘 완수했고, FinOps 관점에서 퍼블릭 클라우드의 가치와 비용 구조를 깊이 있게 고민해 볼 수 있었던 뜻깊은 프로젝트였습니다.

유준형



테라폼으로 프로비저닝된 인프라 위에 Ansible로 클러스터를 올리면서, 수작업 없이 재현 가능한 환경을 만드는 게 뭔지 몸으로 배웠습니다. 손이 많이 갔지만 그만큼 남는 게 많은 프로젝트였습니다.
그리고, 재밌었습니다!

조민석



인프라 구축과 장애 복구를 직접 경험하며 안정성의 중요성을 배울 수 있었고, 팀원들과의 원활한 협업 덕분에 프로젝트를 성공적으로 마무리할 수 있었습니다.

이주희



여러 외부 시스템 (API, redis, DB)을 연동할 때 하나를 실패해도 전체가 죽지 않게 만드는 설계가 중요하다는 것을 깨달았고 로컬 환경에서 먼저 검증하고 실제 서버로 넘어가는 단계적 접근이 중요하고 효율적이라는 것을 배울 수 있었습니다.

이주원



2차 팀 프로젝트를 통해 하이브리드 클라우드 환경에 대해 공부할 수 있어서 좋았고 정말 좋은 팀원들과 함께 편하게 커뮤니케이션을 하면서 자신감을 얻을 수 있는 좋은 시간이였습니다!

Team Endive의 프로젝트 발표를 마칩니다.

경청해 주셔서 감사합니다.